

# Python Sheet Solutions

Earlier due to shortage of time, AI generated solution of dictionary sheet was provided, sorry for the inconvenience caused, here is the updated sheet solution. Peace 🙌

---

## Solution Sheet 7

1.

```
A=[2,6,3,8,2,3,8,8]
```

```
freq_dict = {}
```

```
for item in A:
```

```
    if item in freq_dict:
```

```
        freq_dict[item] += 1
```

```
    else:
```

```
        freq_dict[item] = 1
```

2.

```
A = [1, 2, 3, 1, 2, 5]
```

```
dic = {}
```

```
for i in A:
```

```
    if i in dic:
```

```
        dic[i] += 1
```

```
    else:
```

```
        dic[i] = 1
```

```
for i in dic:
```

```
    if dic[i] == 1:
```

```
        print(i)
```

```
        break
```

3.

```
A = "aabb"
```

```
dic = {}
```

```
for i in A:
```

```
    if i in dic:
```

```
        dic[i] += 1
```

```
    else:
```

```
        dic[i] = 1
```

```
for i in dic:
```

```
    if dic[i] % 2 != 0:
```

```
        print(0)
```

```
        break
```

```
    else:
```

```
        print(1)
```

```
        break
```

4.

```
A = [10, 5, 3, 4, 3, 5, 6]
```

```
dic = {}
```

```
for i in A:
```

```
    if i in dic:
```

```
        dic[i] += 1
```

```
    else:
```

```
        dic[i] = 1
```

```
for i in dic:
```

```
    if dic[i] == 2:
```

```
        print(i)
```

```
        break
```

5.

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}
dict2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}
dict1.update(dict2)
print(dict1)
```

6.

```
dict1 = {'a': 100, 'b': 400, 'c': 300}
target = 400
if target in dict1.values():
    print("found")
```

7.

```
person = {"name": "Nikhil", "age": 19}
for keys, values in person.items():
    print(keys)
```

8.

```
person = {"name": "Nikhil", "age": 19}
for keys, values in person.items():
    print(values)
```

9.

```
person = {"name": "Nikhil", "age": 19}
for keys, values in person.items():
    print(keys, values)
```

10.

```
person = {"name": "Nikhil", "age": 19}
person.clear()
```

```
print(person)
```

11.

```
person = {"name": "Nikhil", "age": 19}
```

```
keys = []
```

```
for key,value in person.items():
```

```
    keys.append(key)
```

```
print(keys)
```

12.

```
person = {"name": "Nikhil", "age": 19}
```

```
values = []
```

```
for key,value in person.items():
```

```
    values.append(value)
```

```
print(values)
```

13.

```
dic = {}
```

```
for i in range(1,6):
```

```
    temp = {i:i**2}
```

```
    dic.update(temp)
```

```
print(dic)
```

14.

```
dict1 = { 1: 'a', 2: 'b', 3: 'c' }
```

```
lis = []
```

```
for key,value in dict1.items():
```

```
    lis.append((key, value))
```

```
print(lis)
```

15.

```

dic = {2:'Apple', 1:'Mango', 3:'Orange', 4:'Banana'}
key = []
for k,v in dic.items():
    key.append(k)
key.sort()
for i in range(len(key)):
    print(key[i],dic[key[i]])

```

16.

```

A = [1,2,2,1]
B = [2,3,1,2]
mydic = {}
N = len(A)
M = len(B)
for i in range(N):
    for j in range(M):
        if A[i] == B[j]:
            if A[i] in mydic.keys():
                mydic[A[i]] +=1
            else:
                mydic[A[i]] = 1
for i,j in mydic.items():
    print('[',(str(i) + ' ') * int(j/2),'],' , end=")

```

17.

```

A = [x**2 for x in range(1,6)]
print(A)

```

18.

```

A = [x for x in range(1,11) if x%2==0 ]
print(A)

```

19.

```
A = [x.capitalize() for x in 'hello']  
print(A)
```

20.

```
a = 4  
print(type(a))  
a = 4,  
print(type(a))  
a = (4)  
print(type(a))  
a = ()  
print(type(a))  
a = (4,)  
print(type(a))  
a = (4,5)  
print(type(a))
```

```
# <class 'int'>  
# <class 'tuple'>  
# <class 'int'>  
# <class 'tuple'>  
# <class 'tuple'>  
# <class 'tuple'>
```

# Solution Sheet 8

1.

```
def sq(n):  
    print(n*n)  
sq(5)
```

2.

```
def area(n):  
    print(n*n)  
area(5)
```

3.

```
def areacircle(n):  
    print((1/2)*n*n*3.14)  
areacircle(5)
```

4.

```
def power(n,m):  
    print(n**m)  
power(5,6)
```

5.

```
def cube(n):  
    print(n**3)  
cube(5)
```

6.

```
def vol(n):  
    print(4*3.14*n*n)  
vol(5)
```

7.

```
def area_ellipse(n,m):  
    print(3.14*n*m)  
area_ellipse(5,6)
```

8.

```
def sums(n):  
    mSum = 0  
    for i in range(n+1):  
        mSum+=i  
    print(mSum)  
sums([1,2,3,4])
```

9.

```
def sqall(n,m):  
    for i in range(n, m+1):  
  
        print(i*i)  
sqall(5,8)
```

10.

```
def summ(n,m):  
    print(n+m)  
summ(5,6)
```

11.

```
def mul(n,m):  
    print(n*m)  
def div(n,m):  
    print(n/m)
```



```

def add(n,m):
    print(n+m)
def sub(n,m):
    print(n-m)

a = input("What do you want to perform: + - * /")
n1 = int(input("N1 = "))
n2 = int(input("N2 = "))
if a == '+':
    add(n1,n2)
elif a == '-':
    sub(n1,n2)
elif a == '*':
    mul(n1,n2)
elif a == '/':
    div(n1,n2)
else:
    print("Error")

```

12.

```

def pyth(a,b):
    c2 = (a*a) + (b*b)
    c = c2**(1/2)
    return c
ans = pyth(3,4)
print(ans)

```

# Solution Sheet 9

1.

```
def introduce(name, age, profession):  
    print(f"Name: {name}, Age: {age}, Profession: {profession}")  
introduce(name="Nikhil", age=19, profession="Student")
```

2.

```
def introduce(name, age, profession="Entrepreneur"):  
    print(f"Name: {name}, Age: {age}, Profession: {profession}")  
introduce("Nikhil", 25)
```

3.

```
def square(x):  
    return x * x  
  
def pythagoras(a, b):  
    return square(a) + square(b)
```

```
c2 = pythagoras(3, 4)  
print("c2 = ", c2)
```

4.

Python

5.

Suyash

Python

Suyash

6.

Error and correction:

There's an error in the code because the variable lang is not defined in the global scope but is being referenced outside the test function. To correct this, define lang globally or pass it as an argument to the test function.

7.

There's an error in the code because the variable name is being referenced inside the fun function before it's defined. To correct this, either define name globally before calling fun or pass it as an argument to fun.

8.

Suyash

Suyash

Chaudhary

Chaudhary

9.

abc

def

def

ghi

ghi

10.

Suyash

11.

Chaudhary

12.

```
sq = lambda x: x**2
```

```
print(sq(4))
```

13.

```
num = int(input())  
even_odd_No = lambda x: 'even' if x % 2 == 0 else 'odd'  
print(even_odd_No(num))
```

14.

```
maxx = lambda x,y: x if x>y else y  
print(maxx(8,5))
```

15.

```
maxx = lambda x, y, z: x if x > y > z else y if y > x > z else z  
print(maxx(8, 5, 2))
```

# Solution Sheet 10

1.

```
import math
print(math.sqrt(100))
print(math.pi)
print(math.sin(2))
print(math.factorial(4))
```

2.

```
import math
print(math.sqrt(16))
print(math.ceil(4.4))
print(math.floor(4.4))
print(math.factorial(5))
```

3.

```
import math
print(math.sqrt(16))
print(round(6.4))
```

4.

```
import random
print(random.randint(0,5))
print(random.random())
print(random.random() * 100)
print(random.choice([1, 4, True, 800, "python", 27, "hello"]))
```

5.

```
import math
r = 4
```

```
area = math.pi * r*r
```

6.

```
import time  
print(int(time.time()))
```

7. import time

```
time.sleep(1)
```

8.

```
import datetime  
print(datetime.datetime.now())
```

9.

```
import random  
numbers = [1, 2, 3, 4, 5, 6]  
random.shuffle(numbers)  
print(numbers)
```

10.

```
from functools import reduce
```

```
arr = [1, 2, 3, 4, 5]  
total_sum = reduce(lambda x, y: x + y, arr)  
print( total_sum)
```

11.

```
import functools
```

```
arr = [1,5,3,8,8]  
maxele = functools.reduce(lambda x, y: x if x > y else y, arr)
```

```
print(maxele)
```

12.

```
import itertools
```

```
arr = [1, 8, 9]
```

```
ele = itertools.accumulate(arr, lambda x, y: x + y)
```

```
print(list(ele))
```

13.

D) `math.fabs()`

14.

D) Returns x raised to the power of y

15.

A) `datetime.now()`

16.

C) `time.time()`

17.

B) Pauses the program execution for the specified number of seconds

18.

A) `random.random()`

19.

B) Returns a randomly selected element from a sequence

# Solution Sheet 11

1.

```
age = 19
```

```
try:
```

```
    result = age / 0
```

```
    print("Result of division:", result)
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero is not allowed.")
```

2.

```
name = "Nikhil Chauhan"
```

```
rollNo = 39
```

```
try:
```

```
    print(name + rollNo)
```

```
except TypeError:
```

```
    print("Error: Cannot concatenate string with integer.")
```

3.

```
def check(num):
```

```
    try:
```

```
        if num % 2 == 0:
```

```
            result = "1" + str(num)
```

```
            print("Concatenated result:", result)
```

```
        else:
```

```
            result = num / 0
```

```
            print("Division result:", result)
```

```
    except ZeroDivisionError:
```

```
        print("Error: Division by zero is not allowed.")
```

```
    except TypeError:
```

```
        print("Error: Invalid operation.")
```



```
check(10)
```

```
check(7)
```

4.

```
try:
```

```
    num1 = float(input("Enter the first number: "))
```

```
    num2 = float(input("Enter the second number: "))
```

```
    result = num1 / num2
```

```
    print("Result of division:", result)
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero is not allowed.")
```

```
except ValueError:
```

```
    print("Error: Invalid input. Please enter numeric values.")
```

5.

```
def concatenate_strings(str1, str2):
```

```
    try:
```

```
        result = str1 + str2
```

```
        print("Concatenated string:", result)
```

```
    except TypeError:
```

```
        print("Error: One of the arguments is not a string.")
```

```
concatenate_strings("Hello", "World")
```

```
concatenate_strings("Hello", 123)
```

6.

```
my_list = [1, 2, 3, 4, 5]
```

```
try:
```

```
    index = int(input("Enter an index to access: "))
```

```
    value = my_list[index]
```

```
    print("Value at index", index, "is:", value)
```

```
except IndexError:
    print("Error: Index out of range.")
except ValueError:
    print("Error: Please enter a valid integer index.")
```

7.

```
try:
    num = int(input("Enter an integer: "))
    print("You entered:", num)
except ValueError:
    print("Error: Please enter a valid integer.")
```

8.

```
class CustomError(Exception):
    pass

try:
    raise CustomError("This is a custom exception.")
except CustomError as e:
    print("Custom exception caught:", e)
```

9.

```
try:
    num1 = int(input("Enter the first number: "))
    num2 = int(input("Enter the second number: "))
    result = num1 / num2
    print("Result of division:", result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Please enter valid numeric values.")
```

except Exception as e:

```
    print("Error:", e)
```

10.

try:

```
    num = int(input("Enter a number: "))
```

```
    if num <= 0:
```

```
        raise ValueError("Error: Entered Negative Numbers")
```

```
    print("You entered:", num)
```

except ValueError as e:

```
    print(e)
```

11-20.

MCQs:

C) To gracefully handle errors that occur during program execution

C) raise

A) Inside the try block

C) ZeroDivisionError

B) It executes regardless of whether an exception occurs or not

B) ZeroError

A) Catches and handles exceptions

C) Using a single try-except block for the entire program

C) It executes only if no exception occurs

C) except (Exception1, Exception2)

# Solution Sheet 12

1.

with open("filename.txt", "r") as file:

```
print(file.read())
```

2.

```
text = input("")
```

with open("output.txt", "w") as file:

```
file.write(text)
```

3.

with open("filename.txt", "r") as file:

```
words = file.read().split()
```

```
print("Word count:", len(words))
```

4.

with open("filename.txt", "r") as file:

```
characters = file.read()
```

```
print("Character count:", len(characters))
```

5.

with open("filename.txt", "r") as file:

```
lines = file.readlines()
```

```
print("Line count:", len(lines))
```

6.

with open("source.txt", "r") as source:

with open("destination.txt", "w") as destination:

```
destination.write(source.read())
```

7. d) All of the mentioned

8. b) `infile.read()`

9. c) `infile.readline()`

10. d) `infile.readlines()`

11. b) a list of lines

12. a) tells you the current position within the file

13. a) `rename(current_file_name, new_file_name)`

14. a) `remove(file_name)`

15. a) sets the file's current position at the offset

16. d) b

17. a) read, append

18. b) Write

19. c) Append

20. c) `Readall()`

21. a) `Readline()`

22. a) `write()`

23. c) `Close()`

24. a) Yes

25. c) `fp.close()`

---

End of Page